Field Hours

CSCE 714 Advanced Hardware Design Functional Verification

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Unit, cluster, and chip

You verify an IP at 3 scales and you do not keep one bench for all of them. The unit is one 32-bit peripheral whose pins you can still drive. The cluster puts several IPs on one bus; new bugs sit in the wiring and the arbiter, not in a unit you closed. The chip adds clocks and pads. A property proved on the unit for 20 cycles is not a cluster proof. You write one goal per scale.

You reuse the unit checker at the cluster instead of rewriting it. The mechanism is an agent bound to an interface, not to a global pin name. The agent that drove the peripheral alone now watches the internal bus. You expect its 4 opcode bins to be hit again by neighbors. If a bin is empty only at the cluster, that opcode never appears. You record whether it is illegal at this scale.

A unit proof does not climb the stack by itself. Each scale has its own goal.

Unit: one IP, all pins

Cluster: several IPs, one bus

Chip: clocks, resets, pads

Reuse the unit agent

An opcode the cluster never sends

1. At the unit, 4 opcode bins are hit by sequences you wrote. A 20-cycle proof covers the unit properties you named.

2. At the cluster the same agent watches the internal bus. Opcode 3's bin stays at 0.

3. Either the neighbors cannot legally send opcode 3, which you waive with a reason, or the wiring dropped it. The unit proof does not decide which.

Problem 1. A property is proved on the unit to a bound of 20 cycles. Does that prove the same property for the cluster?

Answer. No. The cluster adds wiring and an arbiter the unit proof did not include. You write a separate goal for the cluster, even if you reuse the unit agent.

Problem 2. The unit hit 4 opcode bins. At the cluster, 1 of those bins stays empty. What are the two honest explanations?

Answer. The opcode is illegal once the IP is integrated, and you waive that bin with a reason, or the

integration never produces it, which is a cluster bug. The empty bin alone does not say which.

Lecture 2 · 90 minutes

Sequence, driver, monitor, scoreboard

You split the bench so each layer does one job. A sequence names a transaction: opcode 2, length 4, and a payload. A driver turns it into pin wiggles. A monitor rebuilds a transaction from pins. A scoreboard compares it with a reference. You do not let the sequence poke pins, and you do not let the driver judge correctness. When the bus grows from 32 bits to 64, you retune the driver and keep the sequence.

You bring the layers up in an order you can debug. First you send 1 directed sequence through the driver and read what the monitor reconstructed. If those two disagree, the bench is lying and the design is not on trial. Then you connect the scoreboard. Then you enable random sequences. A team that starts with 100 seeds cannot tell a driver bug from a design bug. The 4 layers are what make a failure point at one place.

The sequence never touches a pin. The driver never decides pass or fail.

Sequence: opcode 2 -> Driver wiggles pins -> Monitor rebuilds it -> Scoreboard compares

One directed sequence first

1. You send one sequence: opcode 2, length 4. The driver wiggles the 32-bit bus. The monitor should reconstruct opcode 2 and length 4.

2. If the monitor reports length 8, you fix the monitor or the driver before you look at the design. The scoreboard is still disconnected.

3. Only after that pair agrees do you connect the scoreboard and then turn on random sequences. Starting with 100 seeds would mix bench bugs with design bugs.

Problem 1. The bus width changes from 32 to 64. Which layer do you retune, and which layer should stay?

Answer. Retune the driver, which turns transactions into pin wiggles. Keep the sequence, which still says opcode 2 and length 4. The sequence does not poke pins.

Problem 2. The monitor reports a different length than the one sequence you sent. Where do you look first?

Answer. At the bench, not the design. The driver and the monitor disagree on 1 directed sequence, so one of them is wrong. Connect the scoreboard only after they agree.

Lecture 3 · 90 minutes

One agent, two configurations

You parameterize the agent with a configuration object so 2 instances are not 2 copies of the source. The object carries the width, 32 or 64, an address mask, and whether the agent is active or passive. An active agent drives. A passive agent only monitors, which is what you want on a port you do not own. You build both from the same source. A pasted copy misses a fix you applied to only 1 of the 2 copies.

You pass the configuration before any sequence runs. A sequence that reads the width can emit a payload of 8 bytes on the narrow bus and 16 bytes on the wide bus without an edit. A literal 32 buried in the driver makes the 64-bit instance wrong, and the wide bins stay at 0. You keep 1 object per instance. Sharing 1 object between 2 agents means a change for the peripheral silently retunes the arbiter monitor.

Same source, two objects. A pasted second agent will miss the next bugfix.

Active, width 32

Drives the peripheral

Payload of 8 bytes

Own config object

Passive, width 64

Only monitors

Payload of 16 bytes

Not a second copy

Eight bytes or sixteen

1. The narrow configuration says width 32. The sequence asks the object and builds a payload of 8 bytes.

2. The wide configuration says width 64. The same sequence builds 16 bytes. You did not edit the sequence.

3. If both agents shared 1 object, setting the peripheral to 32 would also force the arbiter monitor to 32. Each instance gets its own object.

Problem 1. Width 32 uses an 8-byte payload and width 64 uses 16 bytes, chosen by the sequence from the configuration. What goes wrong if the driver contains a literal 32?

Answer. The 64-bit instance still behaves as a 32-bit bus. Wide bins stay at 0. The sequence cannot correct a width that the driver ignored.

Problem 2. Why is a passive agent not a second copy of the active source?

Answer. Both are the same source, selected by the configuration: active drives, passive only monitors. A pasted copy would miss a fix applied to only 1 of the 2 files.

Lecture 4 · 90 minutes

Hits, holes, and waivers

You close coverage against a written plan, not a feeling about seeds. A cross of 4 opcodes and 2 directions has 8 bins, and 2 are empty after 50 seeds. Closure means every bin is hit or waived. A waiver names the bin and the reason. Hard to hit is not a reason. If opcode 3 with direction 1 is forbidden by a configuration bit, the denominator becomes 7, not 8.

You aim stimulus at a hole instead of adding seeds. If a seed hits the empty bin 1 time in 100, then 50 seeds are a weak bet. You add a directed sequence that forces opcode 3 and direction 1, run it once, and watch the bin go from 0 to 1. Randomness filled 6 bins. The other 2 are the closure work. You recheck after a design change so a hit bin does not return to 0.

An empty bin is a task. A waiver changes the denominator only with a reason you can read.

1. List the 8 bins

2. See that 2 are empty

3. Force the pair, or waive

4. Denominator may become 7

Two holes after fifty seeds

1. The cross has 4 times 2 = 8 bins. After 50 seeds, 6 bins are nonzero and 2 are 0.

2. One empty bin is illegal under the configuration. You waive it, name the bit that forbids it, and the denominator becomes 7.

3. The other empty bin is legal. You add one directed sequence that forces that pair. Its count goes from 0 to 1. More copies of the same 50 seeds are not the plan.

Problem 1. A cross has 8 bins, 6 are hit, and 1 empty bin is illegal under a configuration bit. What is the denominator after a proper waiver?

Answer. 7. You remove the illegal bin from the plan and you write the reason. The remaining empty legal bin is still open. Hard to hit is not a waiver.

Problem 2. An empty legal bin is hit by about 1 seed in 100, and you have run 50. Why is another 50 of the same stimulus a weak plan?

Answer. Fifty seeds are fewer than the 100 you would expect to wait, and you can still miss. A directed sequence or a constraint that forces the pair hits the bin once, on purpose.

Lecture 5 · 90 minutes

Seeds and one signature

You run the suite on every change, and you record the seed. A failure stores the seed, here 17, the test name, and a signature of the log. Rerunning seed 17 reproduces the same draws, which is the only reason a random failure is debuggable. Lose the seed and the night of 100 seeds is a story you cannot replay. You triage by signature: 12 failures with 1 signature are 1 bug, not 12.

You rank signatures before you ignore any of them. A signature that shows up on 2 tests after a one-line arbiter edit is your first read. A signature that has failed for 9 nights is old, and you do not let it hide a new signature that failed once. You keep the last passing change beside the first failing one. The difference is where you read. A suite that always fails teaches you nothing about today's edit.

The seed replays the draws. The signature groups failures so you debug once.

Save seed 17 -> Hash the failing log -> Group 12 into 1 bug -> Diff the last pass

Twelve logs, one bug

1. Seed 17 fails. You store 17, the test name, and a signature of the log. A rerun with seed 17 hits the same draw.

2. That night, 12 tests fail and all 12 share the signature. You debug 1 bug, not 12.

3. Nine older nights already carry a different signature. You do not start there if a new signature appeared after the arbiter edit. The passing change just before this one is the diff you read.

Problem 1. Twelve failing tests share one signature, and the failure was first seen with seed 17. How many bugs do you open, and how do you replay?

Answer. One bug. You rerun seed 17 to reproduce the draws. Without the seed, the 100-seed night cannot be replayed. The signature is what grouped the 12 logs.

Problem 2. A signature has failed for 9 nights, and a new signature failed once after today's edit. Which do you read first, and why?

Answer. The new signature. The 9-night failure is old and will hide today's bug if you only stare at it. You also keep the last passing change next to the first failing one.

Lecture 6 · 90 minutes

A proof with a small cone

You use a formal tool where the state is small, and you leave the long traces to simulation. A credit counter whose maximum is 4 is such a block. You assert it never goes below 0 and never above 4. A failure comes back as a waveform, here 12 cycles. You turn that waveform into a directed test so the bug stays in the suite after you fix it.

You do not submit the whole chip. The cone of the assertion is the counter and its 2 inputs, not the processor. Assumptions about the handshake are part of the proof. If cluster tests violate an assumption 1 time in 50, the proof does not apply there. You fix the neighbor or weaken the assumption and prove again. A bound of 20 cycles says nothing about cycle 21.

Formal exhausts a small cone. Simulation still owns the chip and the long traces.

Formal cone

Counter max is 4

Proof or a 12-cycle wave

Assumptions are in the claim

Chip simulation

Whole chip, many cycles

May break an assumption

Keeps the directed test

Credits reach five

1. The counter's maximum is 4. The checker asks whether credits can hit 5 or go below 0.

2. The tool returns a waveform of 12 cycles in which the count becomes 5. You add that waveform as a directed test.

3. The cone was the counter and 2 inputs. If you had assumed a handshake the cluster breaks 1 time in 50 tests, the proof does not cover those tests until you fix the assumption.

Problem 1. A credit counter must stay between 0 and 4, and the tool returns a 12-cycle waveform where the count is 5. What do you add to the simulation suite?

Answer. A directed test taken from that waveform, so the bug remains covered after the fix. The formal run found it. The simulation suite is what keeps it from returning unnoticed.

Problem 2. You proved the counter only under a handshake assumption, and cluster tests violate that assumption 1 time in 50. What is the proof worth on those tests?

Answer. It does not apply. The assumption is part of the claim. You fix the neighbor or weaken the assumption and prove again. A bound of 20, if that was the request, also says nothing about cycle 21.

Lecture 7 · 90 minutes

Rules between two IPs

You put a protocol checker on the bus between IPs, because integration bugs are protocol bugs. A valid-ready handshake has 2 rules. While valid is high and ready is low, data must hold. After valid rises it stays high until a cycle where ready is also high. If valid drops after 1 cycle with ready still low, the consumer can miss the beat, and the checker flags that cycle. A payload scoreboard never sees a withdrawn beat.

You check credits the same way. A sender with 4 credits may issue 4 beats, then must wait. Each accepted beat returns 1 credit, and the count stays between 0 and 4. Two IPs that passed unit tests can still deadlock if each waits for the other's first credit. The cluster checker sees that wait. The unit benches never ran the pair. You fail on the illegal cycle, not after a 10000-cycle timeout that only says hung.

The checker names the cycle. A timeout of 10000 only says the test stopped making progress.

1. Valid holds until ready

2. Data stays while waiting

3. Credits stay within 0 to 4

4. Fail on the bad cycle

Valid drops too early

1. Valid rises and ready stays low. The data must hold, and valid must stay high.

2. On the next cycle valid falls while ready is still low. The consumer can miss the beat. The checker fails on that cycle.

3. A sender that has used all 4 credits and issues a fifth beat fails the credit rule the same way. Waiting 10000 cycles to announce a hang does not name the cycle.

Problem 1. Valid is high for 1 cycle, ready stays low, and valid then falls. Which rule fails?

Answer. Valid must stay high until a cycle where ready is also high. Dropping it while ready is low lets the consumer miss the beat. The payload scoreboard may never see that withdrawn beat.

Problem 2. A sender has 4 credits and issues a fifth beat before any credit returns. What do you report, and why is a 10000-cycle timeout worse?

Answer. The credit count left the range 0 to 4, so you fail on that beat. A timeout of 10000 cycles only says the test hung. It does not name the illegal cycle, and two unit-clean IPs can still deadlock as a pair.

Lecture 8 · 90 minutes

Reuse, then close

You can now verify an IP and reuse the work on the next one. You bind one agent, active or passive, to a port of width 32 or 64. You drive sequences, not pins. You close a cross of 8 bins by a hit or a waiver on each bin. You can replay a failure from seed 17, and you can turn a 12-cycle waveform of a 4-credit counter into a directed test.

You can also pick the scale. A unit proof, even to 20 cycles, does not close the cluster, and a cluster protocol checker does not replace the unit scoreboard. When 2 bins are empty you change the stimulus or you waive with a reason. You do not only add seeds. When 12 failing tests share a signature, you debug once. You can write the plan for a new peripheral as reused pieces plus the holes that are actually new.

The plan for a new IP is the reused machinery plus the holes that are actually new.

One agent, two widths

8 bins hit or waived

Seed 17 replays

Formal wave becomes a test

A waveform and two empty bins

1. Formal analysis of a counter whose maximum is 4 returns a 12-cycle waveform in which credits hit 5. You add that waveform as a directed test.

2. The cluster cross has 8 bins and 6 are hit. The new test does not fill the other 2 unless those bins are the same behavior.

3. You still hit or waive each of the 8 bins. The proof and the cross are two ledgers. A unit bound of 20 cycles does not close either of them by itself at the cluster.

Problem 1. A formal run on a counter whose maximum is 4 returns a 12-cycle waveform where credits hit 5. The cluster cross has 6 of 8 bins hit. What do you add to simulation, and does that close coverage?

Answer. You add a directed test from the 12-cycle waveform so the credit bug stays covered. That test does not fill the 2 empty bins unless they are the same behavior. Each of the 8 bins still needs a hit or a waiver. The formal result and the cross are separate.

Problem 2. You have one agent configured for width 32 and you need the same checker on a 64-bit port. What do you reuse, and what must not be shared?

Answer. You reuse the agent source and pass a second configuration with width 64, passive or active as needed. You do not share one configuration object between the two instances, and you do not paste a second copy of the source.